

Vulnerability Assessment and Reporting Engine (VARE)

Submitted by: Muhammad Ali Imran, Laaibah Qayyum , Muhammad Shaayan
Department of Cyber Security
FAST NUCES Islamabad – 2025

Contents

Vulnerability Assessment and Reporting Engine (VARE).....	1
1. Introduction.....	3
2. System Overview	3
3. Architecture & Components	3
4. Machine Learning Workflow	4
5. Vulnerability Processing	4
6. Web Interface & UX	4
7. Sample Output (from <code>static/results/</code>).....	6
8. Evaluation and Performance	7
9. Limitations	7
10. Future Work.....	7
Conclusion.....	7

1. Introduction

- The digital world is increasingly vulnerable to cyber threats due to rapid software development and deployment.
 - Manual vulnerability assessment is resource-intensive and error-prone.
 - VARE is developed to automate this process, improve accuracy, and reduce assessment time.
 - The system integrates static analysis, machine learning, and a web interface to provide a seamless user experience.
-

2. System Overview

- VARE is built using Python and Flask for the backend and HTML/CSS for the frontend.
 - It accepts scan files (JSON format), processes them, and generates a detailed vulnerability report.
 - Core functionalities include file upload, ML-based analysis, result display, and scan history.
 - Designed for modularity to allow future upgrades like real-time scanning and database integration.
-

3. Architecture & Components

Frontend

- Built using HTML templates for different views:
 - `index.html`: Home page with basic info and navigation.
 - `scan.html`: Upload interface to initiate scanning.
 - `results.html`: Displays scan output with vulnerabilities.
 - `history.html`: Shows all past scan records.
- Clean and responsive design.

Backend

- Written in `app.py` using Flask.
- Routes manage user requests and interaction between frontend and ML logic.
- Handles file uploads and triggers the ML classification pipeline.

Utilities

- `scanner.py`: Extracts key data points from uploaded files.

- `vulnerability_processor.py`: Transforms raw data into a format suitable for model input.

Models

- ML components stored in `models/` directory:
 - `vulnerability_model.pkl`: Trained classifier model.
 - `tfidf_vectorizer.pkl`: Converts text into numerical vectors.
 - `label_encoders.pkl`: Encodes labels into machine-readable format.
- These models predict vulnerability severity and categories.

Results & Storage

- Each scan result is stored as a JSON file in `static/results/`.
 - Files are named using timestamps to prevent conflicts and ensure traceability.
-

4. Machine Learning Workflow

- Uses a **TF-IDF Vectorizer** to process textual data into numerical features.
 - The vectorized data is classified using a supervised ML model (likely SVM or Random Forest).
 - Encoders are applied to convert categorical predictions into human-readable labels.
 - The pipeline outputs vulnerability name, severity level, and suggestions.
-

5. Vulnerability Processing

- Uploaded scan results are parsed to extract relevant vulnerability indicators.
 - Data is cleaned and tokenized.
 - Features include vulnerability description, file names, risk level, etc.
 - Output is sorted by severity to help users focus on critical issues first.
-

6. Web Interface & UX

- Designed for simplicity and efficiency:
 - Upload scan button.
 - Dynamic display of classified results.

- History tab to review previous analyses.
- Suitable for security analysts, developers, and IT administrators.

AI Vulnerability System

Home Scan History

Network and OS Vulnerability Scan

Target IP/Range:
127.0.0.1
Examples: Single IP (192.168.1.1), IP range (192.168.1.1-10), CIDR (192.168.1.0/24)

Scan Type:
☐ Quick Scan (Common ports only)
☐ Basic Scan (Port scan with OS detection)
☒ Comprehensive Scan (Vulnerability detection - takes longer)

Start Scan

Scanning in progress...

Finalizing results

Activate Windows
Go to Settings to activate Windows.

AI Vulnerability System

Home Scan History

AI-Driven Vulnerability Prioritization System

This system scans networks and operating systems for vulnerabilities, then uses AI to prioritize them and suggest remediation strategies.

Network & OS Scanning

Scan your network and systems to identify vulnerabilities.

Start Scanning

View Scan History

Browse and analyze previous vulnerability scan results.

View History

System Information

View information about your current system.

System Info

Activate Windows
Go to Settings to activate Windows.

Scan History		
Date/Time	Scan File	Actions
result 20250501	scan_result_20250501_150738.json	View
result 20250501	scan_result_20250501_150810.json	View
result 20250501	scan_result_20250501_150902.json	View
result 20250503	scan_result_20250503_135920.json	View
result 20250503	scan_result_20250503_135953.json	View
result 20250503	scan_result_20250503_141607.json	View
result 20250503	scan_result_20250503_141725.json	View
result 20250503	scan_result_20250503_142025.json	View
result 20250503	scan_result_20250503_142117.json	View

7. Sample Output (from static/results/)

- Example result structure:

Scan type

Quick

Timestamp

20250504_070753

Hosts Summary

Host	Status	OS	Open Ports	Vulnerabilities
127.0.0.1	Up	Unknown	5	0

Host: 127.0.0.1

Open Ports (5)

No vulnerabilities identified for this host.

AI-Driven Vulnerability Prioritization and Remediation System

Activate Windows
Go to Settings to activate Windows.

- All results include:
 - Vulnerability name
 - Severity level
 - Suggested remediation

8. Evaluation and Performance

- Accuracy of ML model tested using labeled vulnerability datasets.
 - High precision in classifying critical vs. non-critical threats.
 - Processing time per scan: typically under 5 seconds.
 - Scalable to larger datasets with minimal latency.
-

9. Limitations

- Relies on the quality of the input scan files.
 - May not detect zero-day vulnerabilities unless trained on relevant data.
 - Lacks integration with external threat intelligence APIs (e.g., CVE databases).
-

10. Future Work

- Add real-time scanning functionality using live network traffic.
 - Incorporate dynamic analysis tools (e.g., Burp Suite, OWASP ZAP).
 - Integrate REST APIs to connect with security dashboards.
 - Enable model retraining via user feedback.
 - Add user authentication and scan scheduling.
-

Conclusion

- VARE automates a critical cybersecurity process—vulnerability assessment.
- Makes use of machine learning to prioritize risks efficiently.
- Empowers organizations to identify and respond to threats faster.
- With additional features and improvements, it has the potential to become a powerful open-source security tool.